AgileFlow AI

File 13 — AI Agent Development Rules

Safe AI-Assisted Coding Rules for the Implementation Team

Project	AgileFlow AI — Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation
Purpose	Define how ChatGPT, Codex, Antigravity, or similar AI coding agents must be used safely during implementation.
Audience	Implementation foundation team, frontend team, backend team, agent component owners, and group leader.
Main Rule	AI can assist development, but humans must review, test, and approve every change before merging.

Quick Summary

- AI agents must not edit random files. Every prompt must clearly define allowed files, protected files, and expected output.
- Shared schemas, API contracts, theme tokens, routes, orchestrator logic, and database collections must not be changed without group approval.
- AI-generated code must be reviewed, tested locally, and committed through a pull request. No direct push to main or develop.
- Secrets, passwords, API keys, database URLs, tokens, and private credentials must never be pasted into an AI prompt or committed to Git.
- Every completed task must update related documentation if APIs, schemas, components, or workflow behavior changed.

1. Purpose of This Document

This document defines the official AI-assisted development rules for AgileFlow AI. The team may use AI tools to speed up implementation, but the project must remain controlled, consistent, secure, and traceable. These rules are designed to prevent duplicated files, missing functions, broken integrations, inconsistent UI, and accidental changes to shared architecture.

2. AI Agent Governance Principles

Principle	Meaning
Human Control	AI suggestions are not automatically accepted. A member must inspect, test, and approve each output.
Bounded Scope	Each AI task must be limited to a specific branch, component, folder, and file list.
No Architecture Drift	AI must not introduce new agents, new workflow stages, or change the approved AgileFlow AI scope.
Traceability First	Outputs should preserve links between requirements, design, tests, code, bugs, security findings, and reports.
Security First	No secrets, tokens, or private data should be given to AI agents or stored in source code.
Documentation First	API, schema, theme, and workflow changes must be reflected in the correct project documents.

3. Approved AI Tool Use Cases

Category	Rule
Allowed	Create boilerplate code, generate simple CRUD endpoints, build reusable UI components, write tests, explain errors, refactor small modules, create mock outputs, generate documentation drafts.
Allowed with Review	Update backend services, implement orchestrator transitions, edit shared components, create schemas, add API integration, implement test execution flow.

Not Allowed Without Approval	Change architecture, rename shared fields, delete files, change workflow stages, change protected routes, add unrelated libraries, modify security or database configuration.
Never Allowed	Expose secrets, paste real credentials, commit private keys, remove human approval gates, bypass traceability, disable validation to make errors disappear.

4. Before Asking an AI Agent to Code

Every member must prepare the following information before sending a coding prompt to an AI agent.

Step	Required Context	Example / Explanation
1	Current branch name	Example: feature/agent-requirements
2	Assigned component	Requirement Gathering, Design, Implementation, Testing, Orchestrator, Reports, etc.
3	Allowed files/folders	Only list files the AI is allowed to modify.
4	Protected files	List files that must not be changed.
5	Relevant documents	API plan, schemas, frontend guide, backend guide, workflow rules.
6	Expected output	API endpoint, UI component, schema, service function, test, report, etc.
7	Validation command	npm run build, npm run lint, pytest, uvicorn run, etc.

5. Standard AI Coding Prompt Contract

Use this structure for most AI coding requests. This keeps AI work bounded and prevents unwanted edits.

```
You are working on AgileFlow AI — Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation.

Component:
[Component name]

Current branch:
[feature branch name]

Allowed files/folders:
- [list exact files/folders]

Do not modify:
- [list protected files/folders]

Follow these documents:
- README.md
- API_CONTRACT.md
- DATA_SCHEMAS.md
- Frontend/Backend guide if relevant
- Orchestrator workflow rules if relevant

Task:
[clear task]
```

Requirements:

- Use existing shared models and naming conventions.
- Do not create duplicate schemas, duplicate APIs, or duplicate components.
- Do not rename existing fields, routes, or statuses.
- Keep implementation simple and readable.
- Add comments only where helpful.
- Return a summary of changed files and how to test.

6. Protected Files and Change Control

The following files should be changed only by the group leader or after group approval. AI agents should not modify them unless the prompt explicitly allows it.

Area	Protected Items
Repository / Docs	README.md, CONTRIBUTING.md, AI_AGENT_RULES.md, API_CONTRACT.md, DATA_SCHEMAS.md, TASK_ASSIGNMENT.md
Frontend Core	frontend/src/App.jsx, frontend/src/main.jsx, frontend/src/routes, frontend/src/layouts/MainLayout.jsx, frontend/src/styles/theme.*
Frontend Shared Components	frontend/src/components/common, frontend/src/components/layout, frontend/src/components/workflow
Backend Core	backend/app/main.py, backend/app/core/config.py, backend/app/database, backend/app/common
Backend Shared Schemas	backend/app/schemas/common, backend/app/schemas/approval, backend/app/schemas/traceability
Orchestrator	backend/app/modules/orchestrator, workflow status definitions, approval transition logic

7. Component Ownership Rules

Component	Branch	Owned Work Area
Requirement Gathering and Analysis Agent	feature/agent-requirements	requirements module, requirement pages, H1 UI, requirement output schemas
Project Management and Design Agent	feature/agent-design-planning	design module, backlog/design pages, H2 UI, design artifact schemas
Implementation and Git Automation Agent	feature/agent-implementation	implementation module, generated code metadata, Git/log UI, H4 UI
Testing, Validation and Security Agent	feature/agent-testing-validation	testing module, pre-code tests, validation UI, bug/security reports, H3/H5 UI
Foundation / Orchestrator	feature/project-foundation or feature/orchestrator	project model, workflow, approvals, shared KB, traceability, integration rules

8. AI Coding Do and Do-Not Rules

Type	Rule
Do	Ask AI to work on one small task at a time.
Do	Tell AI exactly which files can be edited.
Do	Ask AI to summarize changed files and testing steps.
Do	Review generated code before running or committing.
Do	Update documentation when contracts change.
Do Not	Ask AI to “fix the whole project” without file limits.
Do Not	Allow AI to rename common fields such as project_id, requirement_id, status, approval_status, traceability_links.
Do Not	Allow AI to create a second theme system, second API client, second router, or duplicate schema files.
Do Not	Merge AI-generated code without running the application.
Do Not	Paste real credentials, database passwords, JWT secrets, API keys, or private client data.

9. Component-Specific Low-Token Prompt Templates

These prompts are intentionally short. Members can paste them into AI coding tools after attaching the relevant documents or providing current file context.

Use Case	Low-Token Prompt
Frontend Layout	Implement the shared React layout for AgileFlow AI. Use only frontend/src/layouts, frontend/src/components/layout, and frontend/src/styles. Do not edit agent pages. Follow the theme guide and reusable component plan. Return changed files and test steps.
Requirement Agent	Implement the Requirement Gathering page and H1 approval UI using existing shared components and API service conventions. Only edit requirements page/module files. Do not change routes, theme, or shared schemas.
Design Agent	Implement the Project Management and Design page with backlog, design artifact sections, and H2 approval. Use existing component patterns. Do not modify requirement or testing modules.
Implementation Agent	Implement the Implementation and Git Automation dashboard with generated file tree, Git metadata, build logs, and H4 approval. Use mock data first. Do not add real GitHub credentials.
Testing Agent	Implement the Testing, Validation and Security screens for pre-code tests, post-code validation, bug reports, security findings, H3, and H5. Do not change implementation generation logic.
Backend API	Create the FastAPI router, service, repository, request schema, and response schema for this module. Follow the backend guide and API contract. Do not edit unrelated modules or shared models.
Orchestrator	Implement only the next-step transition for the given workflow stage. Follow H1-H5 workflow rules. Do not change approved stage names or status values. Add workflow event logging.

10. Security and Privacy Rules for AI Usage

Area	Rule
Secrets	Never paste or commit database URLs, passwords, API keys, GitHub tokens, JWT secrets, cloud keys, or private keys. Use .env files and .env.example.
Client Data	Use sample data only. Do not paste real private client documents, personal data, or confidential company data into AI tools.
Generated Code	Check for hardcoded secrets, insecure CORS, unsafe file handling, and missing validation before committing.
Dependencies	Do not let AI add many packages unnecessarily. Every new package must have a reason and should be approved by the team.
Logs	Do not log secrets, tokens, credentials, or private uploaded content. Logs should contain workflow, error, and validation information only.

11. AI Output Review Checklist

Before committing AI-generated work, the responsible member must complete this checklist.

Check Area	Question
Scope	Did AI only edit allowed files?
Architecture	Did AI preserve the approved AgileFlow AI architecture and agent responsibilities?
Naming	Are route names, schema fields, statuses, and component names consistent?
Duplicates	Did AI avoid duplicate components, schemas, services, and API clients?
Security	No secrets, unsafe hardcoded values, or private data included?

Frontend	Does the UI use centralized theme tokens and reusable components?
Backend	Does the API follow router/service/repository and response format rules?
Workflow	Are approval gates, workflow events, and traceability links updated where needed?
Testing	Was the app run locally and checked?
Docs	Were related documents updated if contracts changed?

12. Required Testing Before Commit

Area	Minimum Required Checks
Frontend	npm install if needed, npm run lint, npm run build, run the dev server, check browser console, verify responsive layout.
Backend	Install requirements, run FastAPI server, check API docs, test endpoint using Swagger/Postman/curl, verify database write/read if applicable.
Integration	Check frontend service calls backend correctly, validate loading/error states, verify workflow status and approval transitions.
AI Review	Read diff manually. Remove unused files, hallucinated imports, dead code, placeholder secrets, and unrelated edits.

13. Commit and Pull Request Rules for AI-Generated Code

- Do not commit huge AI-generated changes without reviewing them file by file.
- Commit message format: component-name: short task description.
- Every pull request must explain what was generated, what was manually changed, and how it was tested.
- Screenshots must be included for UI changes.
- The PR reviewer must check protected files and ensure no unrelated files were changed.

Item	Rule
PR Title	[Component Name] Short task description
Required Sections	What was added, files changed, APIs changed, schemas changed, testing steps, screenshots, known issues.
Merge Rule	Merge only after local run, code review, and no conflict with API/schema/theme/workflow documents.

14. Common AI-Agent Mistakes to Prevent

Mistake	What to Watch For
Duplicate API client	AI creates a new api.js instead of using existing apiClient.js.
Duplicate components	AI creates a new StatusBadge inside one page instead of using shared StatusBadge.
Schema drift	AI changes requirement_id to reqId or project_id to projectId without approval.
Workflow bypass	AI moves directly from one stage to the next without approval gate.
Uncontrolled styling	AI uses random colors instead of centralized dark purple theme tokens.
Security leak	AI hardcodes MongoDB URL, GitHub token, API key, or password.
Over-engineering	AI adds Redux, GraphQL, microservices, or unnecessary libraries without group decision.

15. Final Acceptance Checklist

- AI task had a bounded prompt with allowed files and protected files.
- No approved architecture, agent responsibility, workflow stage, or approval gate was changed accidentally.
- No secrets or private data were included.

- No duplicate components, schemas, routers, services, or API clients were created.
- Code follows frontend/backend guides and centralized theme/component rules.
- Application was run and tested locally.
- Pull request includes changed files, testing evidence, and screenshots if UI changed.
- Related documentation was updated if APIs, schemas, workflow, or UI contracts changed.

16. Final Rule for the Team

AI can generate code, but the team owns the system. Every AI output must be reviewed, tested, documented, and approved before it becomes part of AgileFlow AI.

Document Control

Field	Value
Version	1.0
Prepared For	AgileFlow AI Implementation Team
Use	Knowledge passing, implementation governance, and AI-assisted development control